

Claude Code mit OpenCode Go

Ein praktisches Backend-Tutorial fuer `claude-o-go`

`claude-o-go`

June 18, 2026

Contents

1	Ziel	3
2	Was vorhanden ist	3
3	<code>claude-o-go</code> von ueberall starten	3
3.1	Einmalige Einrichtung	3
3.2	Warum das funktioniert	4
3.3	Optionale Shell-Konfiguration	4
3.4	Taegliche Nutzung	4
4	Warum ein Proxy noetig ist	4
5	Modell-Kompatibilitaet	5
6	Request Flow	5
7	Wichtige Dateien	6
7.1	<code>claude-o-go</code>	6
7.2	<code>opencode-go-claude-proxy.mjs</code>	6
8	Lokale Checks	7
9	Verifikation	7
10	Haeufige Probleme	7
10.1	Selected model issue	7
10.2	Doppeltes <code>/v1/v1/messages</code>	8

10.3 Claude Code zeigt sonnet	8
11 Mental Model	8

1 Ziel

Dieses Tutorial erklart den Aufbau dieses Repositories und warum er funktioniert. Ziel ist, die Claude Code CLI weiter zu verwenden, aber Modell Anfragen an OpenCode Go statt an Anthropic Claude zu senden.

Claude Code bleibt der Harness: Terminal UI, Tool Calling, Dateioperationen, Permissions, Prompt-Verarbeitung und Agent Loop. OpenCode Go ist das Backend, das die eigentliche Modelanfrage erhaelt.

2 Was vorhanden ist

Dieses Verzeichnis enthaelt:

- `.env`: dein OpenCode Go API Key.
- `claude-o-go`: das Launcher-Skript, das du statt `claude` startest.
- `opencode-go-claude-proxy.mjs`: der lokale Proxy zwischen Claude Code und OpenCode Go.
- `opencode-go-claude-example.mjs`: ein direkter API-Test ohne Claude Code.

Der normale Start:

```
cd /home/user/Desktop/opencode-go-w-claude
./claude-o-go
```

Ein einzelner Prompt:

```
./claude-o-go -p "Reply with exactly: OK"
```

3 claude-o-go von ueberall starten

Du musst nicht jedes Mal in dieses Repository wechseln und es nicht in jedes Projekt kopieren. Installiere es einmal und verwende `claude-o-go` aus jedem Projektverzeichnis.

3.1 Einmalige Einrichtung

Launcher ausfuehrbar machen, falls noetig:

```
chmod +x /home/user/Desktop/opencode-go-w-claude/claude-o-go
```

Symlink in einem Verzeichnis auf deinem PATH anlegen:

```
mkdir -p ~/.local/bin
ln -s /home/user/Desktop/opencode-go-w-claude/claude-o-go \
    ~/.local/bin/claude-o-go
```

Sicherstellen, dass ~/.local/bin auf dem PATH liegt:

```
export PATH="$HOME/.local/bin:$PATH"
```

Shell neu laden:

```
source ~/.bashrc    # oder: source ~/.zshrc
```

3.2 Warum das funktioniert

Der Launcher loest seinen eigenen Symlink mit `readlink -f` auf. Danach wechselt er in das echte Installationsverzeichnis, in dem `.env` und `opencode-go-claude-proxy.mjs` liegen. Dadurch funktioniert der Launcher egal, aus welchem Verzeichnis du ihn aufrufst.

3.3 Optionale Shell-Konfiguration

Wenn du den Key nicht in `.env` halten willst, exportiere ihn einmal in deinem Shell-Profil:

```
export OPENCODE_GO_API_KEY="sk-..."
export OPENCODE_GO_MODEL="qwen3.7-plus"
```

Der Launcher nimmt `OPENCODE_GO_API_KEY` aus der Umgebung und ueberspringt dann `.env`. Das Modell ist nicht mehr im Launcher hartcodiert; setze `OPENCODE_GO_MODEL` im Shell-Profil, damit es ueberall gilt.

3.4 Taegliche Nutzung

Aus einem beliebigen Projekt:

```
cd /path/to/any/project
claude-o-go
```

Ein einzelner Prompt:

```
claude-o-go -p "Explain this repository in one paragraph."
```

Modell fuer einen Lauf wechseln:

```
OPENCODE_GO_MODEL=minimax-m3 claude-o-go
```

4 Warum ein Proxy noetig ist

OpenCode Go stellt mehrere Modelle ueber eine Anthropic-kompatible Messages API bereit. Es ist eine Routing- und Abrechnungsschicht, kein Modell-Host. Jeder Request wird an die API des jeweiligen Modellanbieters weitergeleitet.

Beispiele:

```
qwen3.7-plus
minimax-m3
qwen3.7-max
```

Claude Code validiert Modellnamen lokal, bevor ein Request abgeschickt wird. Ein direkter Start mit einem OpenCode-Go-Modell wird deshalb abgelehnt:

```
ANTHROPIC_BASE_URL=https://opencode.ai/zen/go/v1 \
ANTHROPIC_API_KEY=... \
claude --model qwen3.7-plus
```

Der Proxy loest das, indem Claude Code lokal ein normales Claude-Alias wie **sonnet** sieht. Der Proxy schreibt den ausgehenden Request dann auf das OpenCode-Go-Modell um.

5 Modell-Kompatibilitaet

Jeder Upstream-Anbieter implementiert seinen Anthropic-kompatiblen Endpoint selbst. Deshalb variiert die Kompatibilitaet. Claude Code sendet einen vollstaendigen Anthropic-Payload mit Tools, Cache Control, Tool Choice und Anthropic-Beta-Headern.

Modell-ID	Anbieter	Funktioniert mit Claude Code Har-ness?
qwen3.7-plus	Alibaba (Qwen)	Ja
minimax-m3	MiniMax	Ja
glm-5.2	Z.ai (Zhipu AI)	Nein; invalid API parameter
glm-5.1	Z.ai (Zhipu AI)	Nein; wie glm-5.2
kimi-k2.7-code	Moonshot AI	Nein; lehnt Tool Function Names ab
deepseek-v4-pro	DeepSeek	Nein; erwartet OpenAI Tool Shape

Modelle desselben Anbieters verhalten sich meist wie das getestete Modell. Ungepruefte Modelle sollten vor produktiver Nutzung getestet werden.

Praktische Empfehlung:

```
export OPENCODE_GO_MODEL="qwen3.7-plus" # oder minimax-m3
```

6 Request Flow

Wenn du Folgendes startest:

```
./claude-o-go -p "Reply with exactly: OK"
```

passiert Folgendes:

1. `claude-o-go` liest den OpenCode Go API Key aus `.env`.

2. Es startet den lokalen Proxy auf `http://127.0.0.1:4141`.
3. Es setzt `ANTHROPIC_BASE_URL=http://127.0.0.1:4141`.
4. Es startet Claude Code mit `claude -bare -model sonnet`.
5. Claude Code sendet einen Request an `/v1/messages`.
6. Der Proxy aendert das Modell im Request Body auf das OpenCode-Go-Modell.
7. Der Proxy leitet an `https://opencode.ai/zen/go/v1/messages` weiter.
8. OpenCode Go liefert die Modellantwort.
9. Der Proxy streamt die Antwort zurueck an Claude Code.

7 Wichtige Dateien

7.1 `claude-o-go`

Der Launcher exportiert API Key, Modell und lokale Base URL und startet dann Claude Code:

```
export OPENCODE_GO_API_KEY="$api_key"
export OPENCODE_GO_MODEL="$model"
export ANTHROPIC_BASE_URL="http://127.0.0.1:${proxy_port}"
claude --bare --model "$claude_model_alias" "$@"
```

Wichtige Defaults und Umgebungsvariablen:

- `OPENCODE_GO_MODEL`: erforderlich; kein hartcodierter Launcher-Default.
- `OPENCODE_GO_PROXY_PORT`=4141.
- `OPENCODE_GO_UPSTREAM_RETRIES`=2.
- `CLAUDE_CODE_MODEL_ALIAS`=sonnet.
- `CLAUDE_CODE_OPENCODE_GO_BARE`=1.

7.2 `opencode-go-claude-proxy.mjs`

Der zentrale Rewrite im Proxy:

```
body.model = model
```

Bei Generierungsrequests streamt der Proxy die Antwort von OpenCode Go zurueck an Claude Code. Er beachtet Node-Stream-Backpressure, bricht den Upstream-Request ab, wenn Claude Code die lokale Verbindung schliesst, wiederholt retrybare Upstream-Verbindungsfehler vor dem Senden der Response-Header und wandelt Upstream-Resets mitten im Stream in ein sauberes Streaming-Error-Event um, statt den Proxy-Prozess zu beenden.

Der Proxy implementiert ausserdem Endpunkte, die Claude Code abfragt:

```
HEAD /  
HEAD /v1  
GET /v1/models  
POST /v1/messages/count_tokens  
POST /v1/messages
```

8 Lokale Checks

Vor dem Push ausführen:

```
npm run check
```

Das prüft die Syntax beider JavaScript-Einstiegspunkte.

9 Verifikation

Starte:

```
./claude-o-go -p "Reply with exactly: OK"
```

Achte auf Proxy-Logs wie:

```
OpenCode Go Claude proxy listening on http://127.0.0.1:4141/v1 -> qwen3.7-plus  
POST /v1/messages  
POST /messages -> 200 as qwen3.7-plus  
OK
```

Die wichtigste Zeile ist:

```
POST /messages -> 200 as qwen3.7-plus
```

Das bedeutet: Claude Code hat an den lokalen Proxy gesendet, der Proxy hat an OpenCode Go weitergeleitet, OpenCode Go hat HTTP 200 geliefert, und das Log zeigt das verwendete Upstream-Modell.

10 Häufige Probleme

10.1 Selected model issue

Nutze den Launcher:

```
./claude-o-go
```

Rufe `claude -model qwen3.7-plus` nicht direkt auf.

10.2 Doppeltes /v1/v1/messages

ANTHROPIC_BASE_URL muss auf die Proxy-Wurzel zeigen:

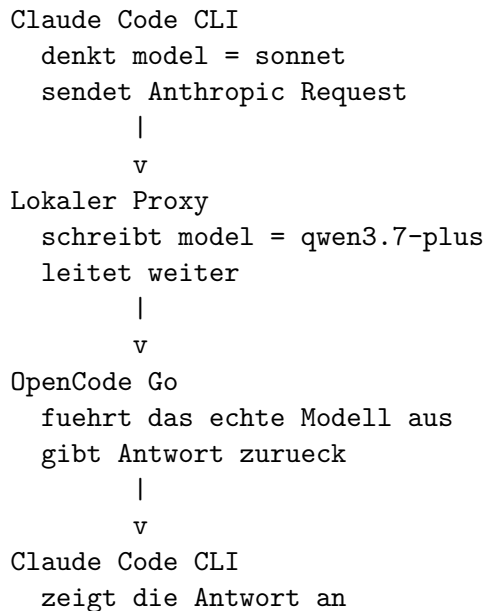
`http://127.0.0.1:4141`

Sie darf nicht /v1 enthalten, weil Claude Code selbst /v1/messages anhaengt.

10.3 Claude Code zeigt sonnet

Das ist erwartet. Claude Code sieht **sonnet**; OpenCode Go erhaelt **qwen3.7-plus**. Das Proxy-Log ist die Quelle der Wahrheit.

11 Mental Model



Die Claude-Code-Nutzung bleibt gleich, aber das Modell-Backend ist OpenCode Go.